COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010 / Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	October 13, 2026	**DATE SUBMITTED:**	October 15, 2026
NAME:	Aguila, Daniel C. Torres, Carl Andrei D.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. HOA12.1 Algorithmic Strategies

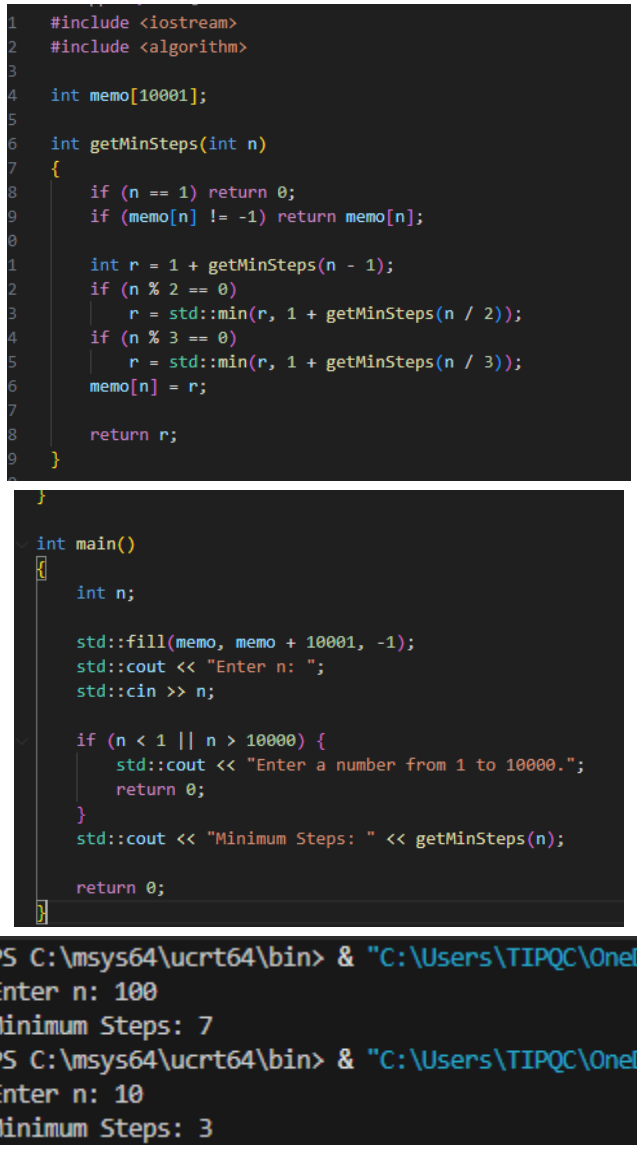
1. PROCEDURE OUTPUT

ILO A: Demonstrate an understanding of algorithmic strategies (Pre-Lab Review)

STRATEGY	ALGORITHM	ANALYSIS
Recursion	**Fibonacci Sequence**	The algorithm **calculates the value of the term using the function that calls itself with reduced values until base values are met**.
Brute Force	**Linear Search Algorithm**	**Linear Search Algorithm** has a **Brute Force Algorithm because it has a function that has a while loop where the loop consists of loops through the array by i<arrSize and checking if dataFind == data[i], which outputs that the user input (std::cin >> toFind) has been successfully been found and immediately exits the function via return**. The **brute force algorithm shows that if the loop continues without finding a match inside the randomly-generated array by the use of the cstdlib for the values of the array and syntax (int i = 0; i<max_size; i++) { dataset[i] = rand(); }, the**

		execution moves past the while loop and outputs that the user input element is not found, and returns the function which shows the brute force algorithm in this Linear Searching Algorithm.
Backtracking	**Depth-First Search (DFS)**	It **tries one direction as much as possible and retreats when it comes across an obstacle**. This way, it can reverse mistaken decisions and take other oaths.
Greedy	**Selection Sort**	The algorithm **continues to select the minimum item and puts it in place**. In this way, **it makes the best possible choice at each stage without changing any previous choices.**
Divide-and-Conquer	**Merge Sort**	**Merge Sort** has a **divide-and-conquer algorithm where the array is separated into subproblems until it separates into individual values**. **After comparing the two individual values in the subproblem**, it will be **merge and compare recursively by the use of a helper function that using a temporary array temp[right - left + 1] to combine them and a primary while loop that compares the values of my left sub-array at i and the right sub-array at j, and two cleanup loops that grab any elements remaining and merge them into one complete and organized array**.

Table 12-1. Algorithmic Strategies and Examples

Screenshot	
Analysis	This solution **makes use of recursion with memoization for solving the Minimum Steps to One problem, where the recurrence relation is F(n) = 1 + min{F(n-1), F(n/2), F(n/3)}**. The **solution will return 0 in the base case when `n == 1`**. The **solution tries all possible moves such as subtracting 1 always and dividing by 2 or 3 only when `n` is divisible by 2 or 3**. This is **done to overcome the limitation of the greedy algorithm in solving some cases, such as 10 where the greedy algorithm takes 4 moves, while this algorithm takes 3 moves**. The **solution utilizes an array called `memo` with all values equal to -1 to store all solved subproblems, so each value is computed once**. This **cuts down the computation time from exponential time to O(n)**.

```cpp
#include <iostream>
#include <algorithm>

int memo[10001];

int getMinSteps(int n)
{
    if (n == 1) return 0;
    if (memo[n] != -1) return memo[n];

    int r = 1 + getMinSteps(n - 1);
    if (n % 2 == 0)
        r = std::min(r, 1 + getMinSteps(n / 2));
    if (n % 3 == 0)
        r = std::min(r, 1 + getMinSteps(n / 3));
    memo[n] = r;

    return r;
}
```

```cpp
int main()
{
    int n;

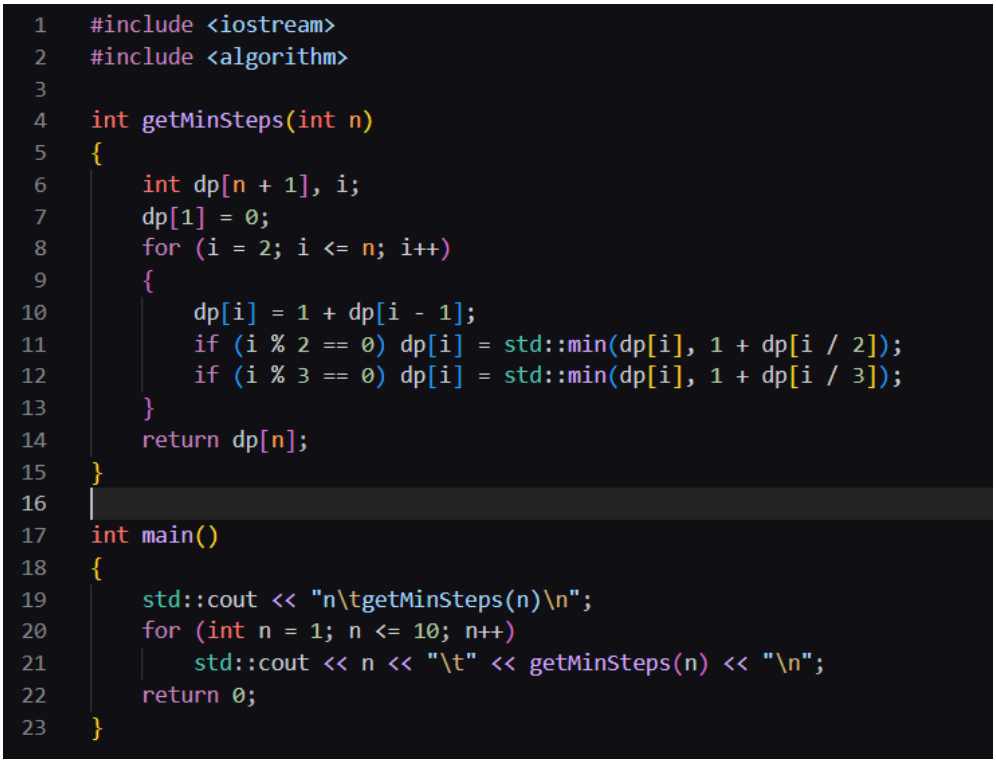
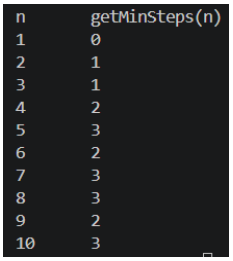
    std::fill(memo, memo + 10001, -1);
    std::cout << "Enter n: ";
    std::cin >> n;

    if (n < 1 || n > 10000) {
        std::cout << "Enter a number from 1 to 10000.";
        return 0;
    }
    std::cout << "Minimum Steps: " << getMinSteps(n);

    return 0;
}
```

```
PS C:\msys64\ucrt64\bin> & "C:\Users\TIPQC\OneDr
Enter n: 100
Minimum Steps: 7
PS C:\msys64\ucrt64\bin> & "C:\Users\TIPQC\OneDr
Enter n: 10
Minimum Steps: 3
```

Table 12-2. Memoization Implementation

Screenshot	 
Analysis	In this code, i've implemented the **bottom-up dynamic programming to find the minimum number of steps to reduce n to 1**, where **each of the step subtracts 1 or divides by 2 or 3**. I've set up the code **(dp[1] = 0) as the base case**, then for each i, I've started from **(1 + dp[i-1]) and used the min to keep the smaller value if the i is divisible by 2 or 3**. **I've implemented the main driver code to get the output console for (n = 1 to 10) is (0, 1, 1, 2, 3, 2, 3, 3, 2, 3)**, which **shows that the values don't always increase, since 6 needs only 2 steps while 5 needs 3**. This shows that the algorithm "Bottom-up dynamic programming has a time complexity of O(n) for best and worst case because of its addition, assignment, two modulo checks, and two min calls with array lookups which none of them depends on n (c * (n-1)) so the time complexity is still O(n).

```cpp
#include <iostream>
#include <algorithm>

int getMinSteps(int n)
{
    int dp[n + 1], i;
    dp[1] = 0;
    for (i = 2; i <= n; i++)
    {
        dp[i] = 1 + dp[i - 1];
        if (i % 2 == 0) dp[i] = std::min(dp[i], 1 + dp[i / 2]);
        if (i % 3 == 0) dp[i] = std::min(dp[i], 1 + dp[i / 3]);
    }
    return dp[n];
}

int main()
{
    std::cout << "n\tgetMinSteps(n)\n";
    for (int n = 1; n <= 10; n++)
        std::cout << n << "\t" << getMinSteps(n) << "\n";
    return 0;
}
```

```
n       getMinSteps(n)
1       0
2       1
3       1
4       2
5       3
6       2
7       3
8       3
9       2
10      3
```

Table 12-3. Bottom-Up Dynamic Programming Implementation

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Create C++ code to implement various algorithmic strategies & ILO C: Create C++ code to solve problems using learned algorithmic strategies

Problem Title: Count the number of paths in a matrix with a given cost to reach the destination cell

Description:
Given an M × N integer matrix where each cell has a non-negative cost associated with it, count the number of paths to reach the last cell (M-1, N-1) of the matrix from its first cell (0, 0) such that the path has given cost. We can only move one unit right or one unit down from any cell, i.e., from cell (i, j), we can move to (i, j+1) or (i+1, j).

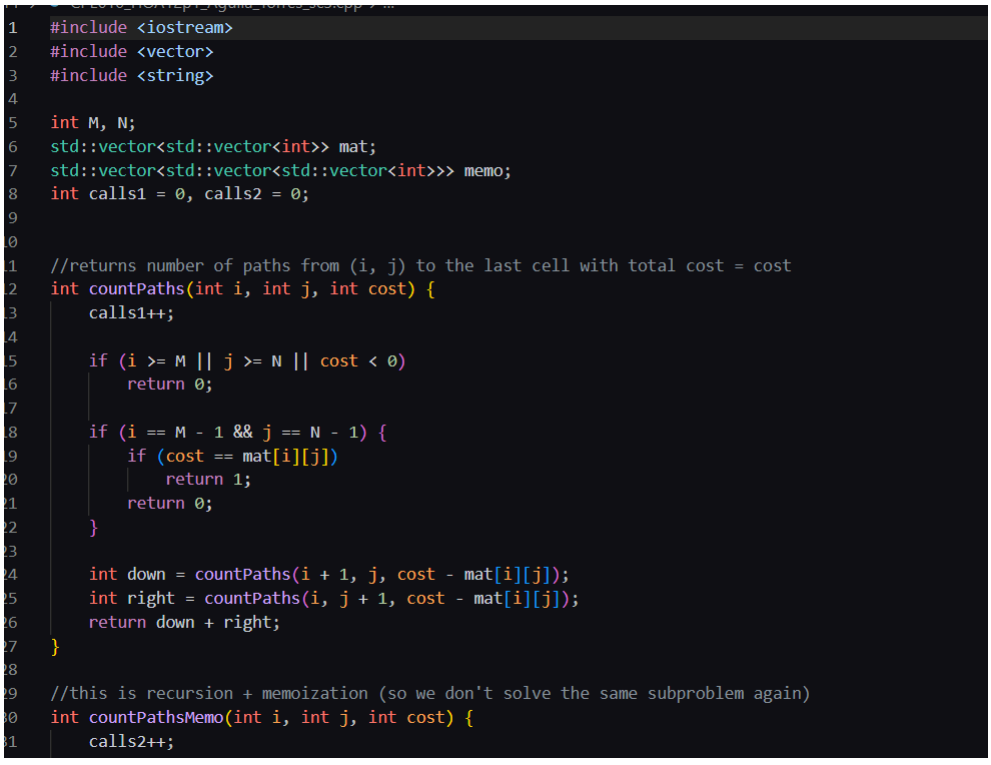
For example, consider the following 4 × 4 matrix where cost is 25. Two paths are having a cost of 25. A sample path has been given when the cost is 25.

4	7	1	6
6	7	3	9
3	8	1	2
7	1	7	3

The idea is to use recursion. The problem has optimal substructure. That means the problem can be broken down into smaller, simple “subproblems”, which can further be divided into yet simpler, smaller subproblems until the solution becomes trivial.

Deliverables:
- Pseudocode (+ Solving Problem by Hand)
- Working C++ Code
- Analysis of working code
- Screenshots of demonstration

Pseudocode (+ Solving Problem by Hand)	countPathsWithCost(matrix, row, col, remainingCost):
	  if the initial position is out of bounds:
	  return 0
	
	  Each function call:
	  subtract the cost of the current cell from remainingCost
	
	  If remainingCost is negative:
	  return 0
	
	  If we have reached the last cell:
	  if remainingCost is 0:
	  return 1
	  else:
	  return 0
	
	  return countPathsWithCost(matrix, row, col + 1, remainingCost)
	  + countPathsWithCost(matrix, row + 1, col, remainingCost)

Working C++ Code	

```cpp
#include <iostream>
#include <vector>
#include <string>

int M, N;
std::vector<std::vector<int>> mat;
std::vector<std::vector<std::vector<int>>> memo;
int calls1 = 0, calls2 = 0;


//returns number of paths from (i, j) to the last cell with total cost = cost
int countPaths(int i, int j, int cost) {
    calls1++;

    if (i >= M || j >= N || cost < 0)
        return 0;

    if (i == M - 1 && j == N - 1) {
        if (cost == mat[i][j])
            return 1;
        return 0;
    }

    int down = countPaths(i + 1, j, cost - mat[i][j]);
    int right = countPaths(i, j + 1, cost - mat[i][j]);
    return down + right;
}

//this is recursion + memoization (so we don't solve the same subproblem again)
int countPathsMemo(int i, int j, int cost) {
    calls2++;
```

```
28
29  //this is recursion + memoization (so we don't solve the same subproblem again)
30  int countPathsMemo(int i, int j, int cost) {
31      calls2++;
32
33      if (i >= M || j >= N || cost < 0)
34          return 0;
35
36      if (i == M - 1 && j == N - 1) {
37          if (cost == mat[i][j])
38              return 1;
39          return 0;
40      }
41
42      if (memo[i][j][cost] != -1)
43          return memo[i][j][cost];
44
45      int down = countPathsMemo(i + 1, j, cost - mat[i][j]);
46      int right = countPathsMemo(i, j + 1, cost - mat[i][j]);
47      memo[i][j][cost] = down + right;
48      return memo[i][j][cost];
49  }
50
51  //dp[i][j][c] = number of paths from (0,0) to (i,j) with total cost c
52  int countPathsDP(int K) {
53      std::vector<std::vector<std::vector<int>>> dp(
54          M, std::vector<std::vector<int>>(N, std::vector<int>(K + 1, 0)));
55
56      if (mat[0][0] <= K)
57          dp[0][0][mat[0][0]] = 1;
```

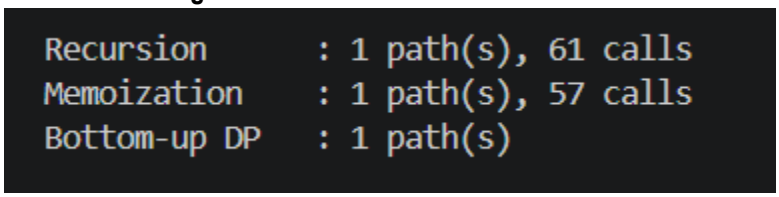
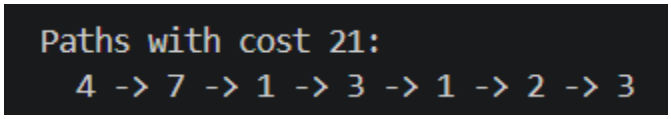
```
56      if (mat[0][0] <= K)
57          dp[0][0][mat[0][0]] = 1;
58
59      for (int i = 0; i < M; i++) {
60          for (int j = 0; j < N; j++) {
61              for (int c = 0; c <= K; c++) {
62                  if (i == 0 && j == 0)
63                      continue;
64                  if (c < mat[i][j])
65                      continue;
66                  int before = c - mat[i][j];
67                  if (i > 0)
68                      dp[i][j][c] += dp[i - 1][j][before];
69                  if (j > 0)
70                      dp[i][j][c] += dp[i][j - 1][before];
71              }
72          }
73      }
74      return dp[M - 1][N - 1][K];
75  }
76
77  //print every path that has the given cost
78  void printPaths(int i, int j, int cost, std::string path) {
79      if (i >= M || j >= N || cost < 0)
80          return;
81
82      path += std::to_string(mat[i][j]);
83
84      if (i == M - 1 && j == N - 1) {
85          if (cost == mat[i][j])
86              std::cout << "  " << path << "\n";
87          return;
88      }
```

```
89
90      path += " -> ";
91      printPaths(i + 1, j, cost - mat[i][j], path);
92      printPaths(i, j + 1, cost - mat[i][j], path);
93  }
94
95  int main() {
96      int K;
97
98      std::cout << "Enter number of rows and columns: ";
99      std::cin >> M >> N;
100
101     mat.assign(M, std::vector<int>(N));
102     std::cout << "Enter the matrix:\n";
103     for (int i = 0; i < M; i++)
104         for (int j = 0; j < N; j++)
105             std::cin >> mat[i][j];
106
107     std::cout << "Enter the target cost: ";
108     std::cin >> K;
109
110     memo.assign(M, std::vector<std::vector<int>>(N, std::vector<int>(K + 1, -1)));
111
112     int r1 = countPaths(0, 0, K);
113     int r2 = countPathsMemo(0, 0, K);
114     int r3 = countPathsDP(K);
115
116     std::cout << "\nRecursion      : " << r1 << " path(s), " << calls1 << " calls\n";
117     std::cout << "Memoization    : " << r2 << " path(s), " << calls2 << " calls\n";
118     std::cout << "Bottom-up DP   : " << r3 << " path(s)\n";
119
120     std::cout << "\nPaths with cost " << K << ":\n";
121     printPaths(0, 0, K, "");
122
123     return 0;
```

int calls1

Generate Copilot summa

Analysis	**Torres:** I've analyzed in the implementations for the supplementary activity, we have developed there distinct algorithm strategies which are the plain recursion (countPaths function), recursion with memoization (countPathsMemo function), and a bottom-up dynamic programming (countPathsDP) which was supported by a printPaths helper function to evaluate the matrix path costs. Also, we've tested a 4x4 matrix against a target cost of 25, which identifies that there is 0 matching paths. On the output console, I've analyzed that it highlights the optimization provided by the memoization technique, which reduced the total function calls from 77 in plain recursion down to 67 by storing the intermediate subproblems. Lastly, the bottom-up DP approach successfully calculated the matrix state iteratively without recursive overhead, while the printPaths function confirms that there are no valid traversal routes met the target criteria. **Aguila:** I've discovered that this algorithm applies recursion to calculate the number of possible routes from the source cell to the destination cell in a two-dimensional array in such a way that the sum of all costs of cells in each route is equal to the specified cost. It makes either right movement or down movement and reduces the cost of the current cell from the remaining cost. When the cell is not within the boundary or the remaining cost becomes less than zero, it returns zero as it is an invalid route. Once it reaches to the destination cell, it returns 1 if the remaining cost equals to zero otherwise returns zero. At last, it calculates the sum of right and down movements and returns it. The time complexity of this algorithm is O(2^(M+N)) as it creates two branches at each step, i.e., one for right movement and another for down movement.

Screenshots of Demonstrations	**Aguila:** **First Test: 21 Target Cost**  **There's only one available path for the target cost of 21**  **Step 1 : (0,0) = 4** **Step 2 : (0,1) = 11** **Step 3 : (0,2) = 12** **Step 4: (1,2) = 15** **Step 5: (2,2) = 16** **Step 6 : (2,3) = 18** **Step 7 : (3,3) = 21**

	Torres:
	Second Test: 25 Target Cost
	
	`Recursion : 0 path(s), 77 calls`
	`Memoization : 0 path(s), 67 calls`
	`Bottom-up DP : 0 path(s)`
	
	There's no available path for the target cost 25
	
	`Paths with cost 25:`
	
	NO STEPS, which means it didn't reach 25

Daniel's Conclusion/Learnings/Analysis	**Summary of lessons learned:** I learned that different algorithmic strategies can be used to solve problems more efficiently depending on their requirements. I also learned how dynamic programming uses memoization and a bottom-up approach to store previous results and avoid repeated calculations. In addition, I learned how recursion can solve problems by breaking them into smaller subproblems, such as counting paths in a matrix with a given cost. **Analysis of procedure:** The procedure helped me understand how each of these algorithms really works. For the problem Minimum steps to one problem, it showed the greedy approach but it fails when it comes to having n = 10 because it takes 4 steps instead of 3. Then I learned how memoization and bottom-up dynamic programming work, and they both got the correct answer, since memoization goes top-down using the saved results while bottom-up starts from the smallest case and builds up. **Analysis of supplementary:** The supplementary activity helped me apply recursion to count the number of paths in a matrix that have a specific total cost. The function checks whether a position is valid, subtracts the current cell's cost, and explores the possible paths to the right and down. This activity helped me understand how base cases and recursive calls work together to solve a problem. **How well did you think you did in this activity?** I think that I have done quite well in this task because I managed to understand each one of the strategies and how recursion can be used to

	solve the extra question. Since our solution produced the same output as our manual solution, the logic must be right. One area that I have to work on is analysis of algorithms and their time and space complexity. Moreover, I also wish to learn to implement recursive functions as well as when to use dynamic programming. I think one area that I have to work on is analysis of algorithms and their time and space complexity but I also wish to learn to implement recursive functions as well as when to use dynamic programming.
Carl's Conclusion/Learnings/Analysis	In this activity, I've learned the different distinct algorithm strategies and how they are being implemented in C++. For example, I've learned that brute force algorithm searches sequentially through the datasets. Also, Divide and Conquer breaks problems into subproblems like the Merge Sort Algorithm. Moreover, Recursion tackles repetitive calculations like the Fibonacci Sequence. Also, I've learned how dynamic programming techniques like top-down memoization and bottom-up DP are being implemented by storing subproblems which optimizes the time complexity down to O(n). In the Supplementary Activity, I've learned how to assess the trade-offs between the execution speed, stack overhead, and the memory allocation across each algorithms that are needed for the problem. Lastly, I've learned that we continue to transition from the Basic Recursion or Greedy Algorithm to a Structured Dynamic Programming like Top-Down Memoization and Bottom-Up Dynamic Programming which affects the time and space complexity of the code itself. I think I did well mostly for the procedures and I still need to improve my implementation on different algorithms like Recursion and Bottoms-Up Dynamic Programming.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used AI just to fix my grammar and choice of words**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

COMPUTER ENGINEERING

QUEZON CITY

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.